paths2d.py reference

Stage 1 of the 2D multipath simulator: geometry to path lists to HDF5. Reference for every public function and class, with worked examples. File format version 1.0. Written 2026-10-18.

Contents

Overview 2

Conventions 2

Constants 2

Scene and geometry 2

- `Scene` . . . 2
- `rectangle_room` . . . 3
- `add_polygon` . . . 3
- `add_box` . . . 3
- `add_wall` . . . 4
- `free_space_mask` . . . 4

Images 4

- `mirror_points` . . . 4
- `ImageSet` . . . 4
- `compute_images` . . . 5

Path tracing 5

- `trace_paths` . . . 5
- `PathSet` . . . 6

Receiver-position helpers 6

- `make_rx_grid` . . . 6
- `make_ula` . . . 7

HDF5 input and output 7

- `save_pathset` . . . 7
- `load_pathset` . . . 7

Plotting 7

- `plot_paths` . . . 8

Internal helpers 8

How the tracer works 8

Example use cases 9

Limits and pitfalls 14

Overview

What it does. Given a 2D floor plan made of line-segment walls, one transmitter and many receiver points, `paths2d.py` finds every specular propagation path (the direct path plus wall bounces up to a chosen order) using the image-source method, and stores the geometric description of each path. It is frequency-independent: no band, bandwidth, material permittivity or reflection coefficient is applied here. Stage 2 (not part of this file) reads the saved paths and builds H(f) for any band without re-tracing.

Pipeline. `Scene` builders, then `trace_paths` (which calls `compute_images` and the per-receiver backtrace), then `PathSet`, then `save_pathset` to HDF5. Analysis code calls `load_pathset`.

Dependencies. `numpy`, `h5py`; `matplotlib` only for `plot_paths`. Everything is in one module: `import paths2d as p2`.

Conventions

Item	Convention
Units	Metres and radians. Speed of light is `C0` = 299,792,458 m/s.
Angles	Measured from the +x axis, counter-clockwise positive.
Walls	Line segments. Two-sided and opaque: they reflect from either face and block all paths; there is no transmission, diffraction or diffuse scattering.
Wall ids	Indices into the Scene arrays, in the order walls were added.
Path ordering	Within a receiver, paths are sorted by increasing length (first arrival first). Wall sequences and bounce points are stored in propagation order: index 0 is the bounce nearest the Tx.
Padding	Wall ids are padded with -1; angles and points are padded with NaN, out to `max_order` columns.
Immutability	Scene builders return a new Scene. Always assign the result: `scene = p2.add_box(scene, ...)`.

Constants

Name	Value	Meaning
`C0`	299792458.0	Speed of light in m/s. Delay = length / C0.
`FORMAT_VERSION`	"1.0"	HDF5 layout version. `load_pathset` rejects a different major number.
`U_TOL`	1e-9	Tolerance on the along-wall parameter (0 to 1) when deciding whether a crossing lies on a wall segment.

Scene and geometry

Scene

```
@dataclass
class Scene:
    p0, p1, material, group, group_kind, material_names
```

The container for all wall segments. Normally created by the builder functions below rather than directly.

Parameter	Type	Meaning
`p0, p1`	(N, 2) float	Start and end point of each wall segment.
`material`	(N,) int	Index into `material_names` for each wall (stored for stage 2; unused in tracing).

Parameter	Type	Meaning
`group`	(N,) int	Group id of each wall. A group is one room outline, one obstacle polygon, or one free-standing wall.
`group_kind`	list of str	"room", "obstacle" or "wall" for each group id. Used by `free_space_mask`.
`material_names`	list of str	Material labels, default ["wall"]. Append your own, e.g. `scene.material_names.append("glass")`, and use the index.

Notes. Properties and methods: `n_walls` (number of segments), `normals()` (unit normals, (N, 2), segment direction rotated +90 degrees), `lengths()` (segment lengths, (N,)).

`rectangle_room`

```
rectangle_room(Lx, Ly, material=0, material_names=("wall",)) -> Scene
```

Creates a rectangular room or hallway with its lower-left corner at the origin. A hallway is just a long, thin room.

Parameter	Type	Meaning
`Lx, Ly`	float	Extent along x and y in metres.
`material`	int	Material index for all four walls.
`material_names`	sequence of str	Initial material label list.

Returns. A Scene with wall ids 0 = bottom (y=0), 1 = right (x=Lx), 2 = top (y=Ly), 3 = left (x=0). Group 0 has kind "room".

`add_polygon`

```
add_polygon(scene, vertices, material=0, kind="obstacle") -> Scene
```

Adds a closed polygon (the last vertex connects back to the first) as one new group of walls.

Parameter	Type	Meaning
`scene`	Scene	Scene to extend. It is not modified.
`vertices`	(V, 2) array-like	Polygon corners in order.
`material`	int	Material index for all edges.
`kind`	str	"obstacle" (default): solid interior, so receivers inside are masked. "room": the polygon is a room outline (use for non-rectangular rooms, see Example 6). "wall": no interior.

Returns. A new Scene.

`add_box`

```
add_box(scene, xmin, ymin, xmax, ymax, material=0) -> Scene
```

Adds an axis-aligned rectangular obstacle (shorthand for `add_polygon` with four corners, kind "obstacle").

Parameter	Type	Meaning
`scene`	Scene	Scene to extend.
`xmin, ymin, xmax, ymax`	float	Box extents in metres.
`material`	int	Material index.

Returns. A new Scene.

`add_wall`

```
add_wall(scene, a, b, material=0) -> Scene
```

Adds a single free-standing wall segment from a to b (a partition or screen). It reflects and blocks from both sides and has no interior, so it does not affect `free_space_mask`.

Parameter	Type	Meaning
`scene`	Scene	Scene to extend.
`a, b`	(2,) array-like	Segment endpoints in metres.
`material`	int	Material index.

Returns. A new Scene.

`free_space_mask`

```
free_space_mask(scene, pts) -> (R,) bool array
```

Tells which points are legal transmitter or receiver locations: inside every "room" group and outside every "obstacle" group, by even-odd ray casting.

Parameter	Type	Meaning
`scene`	Scene	The scene.
`pts`	(R, 2) array-like	Points to test.

Returns. Boolean mask, True where the point is legal.

Notes. Points exactly on a wall are not classified reliably. `trace_paths` calls this for you.

Images

`mirror_points`

```
mirror_points(pts, a, b) -> (M, 2) array
```

Mirrors points across the infinite line through a and b. The building block of the image method.

Parameter	Type	Meaning
`pts`	(M, 2) or (2,) array-like	Points to mirror.
`a, b`	(2,) array-like	Two distinct points on the mirror line.

Returns. The mirrored points, shape (M, 2).

`ImageSet`

```
@dataclass
class ImageSet:
    pos, order, walls, parent
```

All image sources of one transmitter. Row 0 is the real source.

Parameter	Type	Meaning
`pos`	(M, 2) float	Image positions.
`order`	(M,) int	Number of reflections that produced each image.
`walls`	(M, max_order) int	Wall sequence in propagation order, padded with -1.
`parent`	(M,) int	Index of the image one reflection earlier; -1 for the real source.

`compute_images`

```
compute_images(scene, tx, max_order) -> ImageSet
```

Builds every image of `tx` up to `max_order` reflections. Each order-n image is the mirror of an order n-1 image across one wall, excluding the wall that made the previous reflection (mirroring twice across the same wall just returns to the previous image). Whether a path physically exists is NOT decided here; that depends on the receiver and is checked in `trace_paths`. The image set depends only on the Tx and the walls, so it is computed once per Tx.

Parameter	Type	Meaning
`scene`	Scene	The walls.
`tx`	(2,) array-like	Transmitter position.
`max_order`	int	Maximum reflections; 0 means the direct path only.

Returns. An ImageSet.

Notes. Raw image count grows as N x (N-1)^(order-1) for N walls. Keep `max_order` modest for scenes with many segments.

Path tracing

`trace_paths`

```
trace_paths(scene, tx, rx_points, max_order, max_length=None,
            eps_len=1e-9, progress=False, meta=None) -> PathSet
```

The main entry point. Finds all valid specular paths from one transmitter to every receiver point.

Parameter	Type	Meaning
`scene`	Scene	The walls.
`tx`	(2,) array-like	Transmitter position (m). Must be in free space, otherwise ValueError.
`rx_points`	(R, 2) array-like	Receiver positions (m). Points in obstacles or outside the room are kept in the output, flagged `rx_valid=False`
`max_order`	int	Maximum reflections. Contributions beyond order 4 to 6 are usually negligible.
`max_length`	float or None	Drop paths longer than this many metres. None keeps all.
`eps_len`	float	Length tolerance (m): wall crossings this close to a leg's endpoint are ignored in the occlusion test.
`progress`	bool	Print coarse progress every 10% of receivers.
`meta`	dict or None	Extra JSON-serializable settings (scene name, seed, ...) stored with the paths.

Returns. A PathSet.

Notes. Invalid receivers get zero paths. `meta` always also records `max_order`, `max_length` and `eps_len`. Measured speed on an 8-wall hallway at order 4: about 3.6 ms per receiver.

`PathSet`

```
@dataclass
class PathSet:
    scene, tx, rx, rx_valid, offsets, length, order, walls,
    inc_angle, points, aoa, aod, meta
```

Flat, ragged storage of all paths for one Tx and many receivers (CSR style). Paths of receiver r occupy rows `offsets[r]:offsets[r+1]` of every per-path array, sorted by length. P is the total number of paths, R the number of receivers, K = max_order.

Parameter	Type	Meaning
`scene`	Scene	The scene that was traced.
`tx`	(2,) float	Transmitter position.
`rx`	(R, 2) float	Receiver positions (m).
`rx_valid`	(R,) bool	False for receivers in obstacles or outside the room (they have no paths).
`offsets`	(R+1,) int64	Row pointers into the per-path arrays.
`length`	(P,) float	Total path length in metres. Delay is length / C0.
`order`	(P,) int32	Number of reflections (0 = direct).
`walls`	(P, K) int32	Wall ids in propagation order, -1 padded.
`inc_angle`	(P, K) float	Angle between the incoming ray and the wall normal at each bounce, radians 0 to pi/2 (0 = normal incidence). NaN padded. Needed for Fresnel coefficients.
`points`	(P, K, 2) float	Bounce coordinates in propagation order, NaN padded.
`aoa`	(P,) float	Angle of arrival: direction the receiver looks to see the arriving ray (pointing from Rx back along the last leg), radians.
`aod`	(P,) float	Angle of departure: direction the ray leaves the Tx, radians.
`meta`	dict	Run settings: max_order, max_length, eps_len, plus anything you passed.

Method	Returns
`n_paths()`	(R,) int array: number of paths per receiver.
`rx_slice(r)`	A Python slice selecting receiver r's rows in every per-path array.
`delays()`	(P,) array: propagation delay of every path in seconds, length / C0.

Receiver-position helpers

`make_rx_grid`

```
make_rx_grid(xlim, ylim, step) -> (pts, xs, ys)
```

Makes a regular receiver grid in metres. Use a metre-based spacing so one trace serves every band; pick lambda/4 subsets later (Example 7).

Parameter	Type	Meaning
`xlim, ylim`	(min, max) tuples	Grid extents in metres.
`step`	float	Grid spacing in metres.

Returns. pts (ny*nx, 2) row-major, x varying fastest (reshape to (ny, nx, 2)); **xs**, **ys** 1D coordinate vectors.

Notes. A degenerate range such as ylim=(1.5, 1.5) gives a single row (a line of receivers).

make_ula

```
make_ula(center, spacing, n, angle=0.0) -> (n, 2) array
```

Makes the element positions of a uniform linear array centred on `center`.

Parameter	Type	Meaning
`center`	(2,) array-like	Array centre (m).
`spacing`	float	Element spacing (m), e.g. wavelength / 2.
`n`	int	Number of elements.
`angle`	float	Array axis direction from +x, radians.

Returns. Element positions, shape (n, 2), ordered along the axis.

HDF5 input and output

save_pathset

```
save_pathset(filename, ps, config=None) -> None
```

Writes a PathSet, its scene and its settings to an HDF5 file (gzip level 4 with shuffle on non-empty datasets). An existing file is overwritten.

Parameter	Type	Meaning
`filename`	str	Output path.
`ps`	PathSet	Paths to save.
`config`	dict or None	Extra JSON-serializable settings (seed, notes, code version), merged with `ps.meta` into the `config_json` attribute.

load_pathset

```
load_pathset(filename) -> PathSet
```

Reads a file written by `save_pathset` and rebuilds the full PathSet, including the Scene and meta.

Parameter	Type	Meaning
`filename`	str	HDF5 file to read.

Returns. A PathSet identical to the one saved.

Notes. Raises ValueError if the file's format major version differs from this module's.

File layout

Location	Contents
/ (attributes)	format_version, created_utc (ISO string), c0, config_json (JSON of merged meta and config)
/scene	p0 (N,2), p1 (N,2), material (N,), group (N,); attributes group_kind_json, material_names_json
/tx	xy (2,)
/rx	xy (R,2), valid (R,) bool, offsets (R+1,) int64
/paths	length (P,), order (P,), walls (P,K), inc_angle (P,K), points (P,K,2), aoa (P,), aod (P,)

Plotting

`plot_paths`

```
plot_paths(ps, rx_index, ax=None) -> matplotlib Axes
```

Draws the walls, the Tx (green triangle), one receiver (red dot) and all of its paths: direct in red, reflections in blue. The legend sits below the axes and the title shows the path count.

Parameter	Type	Meaning
`ps`	PathSet	Paths to draw.
`rx_index`	int	Row index into `ps.rx` (not a coordinate; find one with np.argmin of distances).
`ax`	Axes or None	Axes to draw on; a new figure if None.

Returns. The Axes used. `matplotlib` is imported lazily inside the function.

Internal helpers

Prefixed with an underscore; not part of the public interface, listed for readers of the source.

Name	Purpose
`_append(scene, p0, p1, material, kind)`	Returns a new Scene with segments added as one new group.
`_empty_scene(material_names)`	A Scene with no walls (the start for custom outlines, Example 6).
`_inside_group(scene, gid, pts)`	Even-odd ray-casting inside test for one group's closed outline.
`_cross2(a, b)`	2D cross product along the last axis.
`_seg_intersect(a, b, c, d)`	Vectorized segment intersection returning parameters t (along a-b), u (along c-d) and a parallel flag; NaN where parallel.
`_leg_blocked(a, b, scene, skip_a, skip_b, eps_len)`	True for legs crossed by any wall other than the walls they start or end on.
`_empty_paths(K)`	Zero-row per-path arrays used when a receiver has no paths.
`_trace_one(...)`	All valid paths from the Tx to one receiver (the algorithm below).
`_ds(group, name, arr)`	Writes an HDF5 dataset, with gzip only when non-empty.

How the tracer works

1. Images. `compute_images` mirrors the Tx across walls recursively, order by order, remembering each image's wall sequence and parent.

2. Backtrace per receiver. For an image of order n with wall sequence (w1 ... wn), start at the Rx. For k = n down to 1: draw the segment from the current point to image k, intersect it with wall wk, require the crossing to lie inside the segment, make it bounce point Pk and continue from there toward the next-earlier image. The last leg runs to the real Tx. The check also guarantees the receiver and the image are on opposite sides of each wall, which is exactly the reflection condition.

3. Occlusion. Every leg is tested against all walls except the ones it starts and ends on. A leg is blocked by a proper crossing in its interior; crossings within `eps_len` of an endpoint are ignored, and crossings at a wall's very end count as blocked (conservative).

4. Output. Surviving paths get length, bounce points, incidence angles and angles of arrival and departure, and are sorted by length.

Duplicate images. In a rectangle, the wall sequences (left, bottom) and (bottom, left) produce the same image position, but only one ordering passes the backtrace for any given receiver, so no path is counted twice. This is verified against the closed-form lattice formula in the test suite.

Cost. Raw images grow as N x (N-1)^(order-1); the per-receiver cost scales with images x walls. Most candidate paths are rejected at the first intersection test.

Example use cases

Every snippet below was executed against this version of `paths2d.py` and the output shown is the real output. Save each as a script next to `paths2d.py`.

Example 1: smallest possible run

One rectangular room, one transmitter, one receiver. Paths come back sorted by length, so index 0 is the first arrival.

```
import paths2d as p2

room = p2.rectangle_room(10.0, 8.0)              # 10 m x 8 m, corner at origin
ps = p2.trace_paths(room, tx=(2.0, 3.0), rx_points=[[8.0, 5.0]], max_order=2)

print("paths:", ps.n_paths()[0])
for L, n in list(zip(ps.length, ps.order))[:5]:   # shortest five
    print(f"order {n}  length {L:6.2f} m  delay {L / p2.C0 * 1e9:6.2f} ns")
```

Output:

```
paths: 13
order 0  length   6.32 m  delay  21.10 ns
order 1  length  10.00 m  delay  33.36 ns
order 1  length  10.00 m  delay  33.36 ns
order 1  length  10.20 m  delay  34.02 ns
order 1  length  10.20 m  delay  34.02 ns
```

Example 2: hallway with an obstacle, receiver grid, save and reload

The standard workflow: build the scene, make a receiver grid, trace, write HDF5, read it back. `meta` and `config` end up in the file so the run is self-describing.

```
import numpy as np
import paths2d as p2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)    # hallway + box
tx = (2.0, 1.5)
rx, xs, ys = p2.make_rx_grid(xlim=(10.0, 11.0), ylim=(0.5, 2.5), step=0.05)

ps = p2.trace_paths(scene, tx, rx, max_order=3, meta={"scene": "hallway_box"})
p2.save_pathset("hallway_box_o3.h5", ps, config={"seed": 1})

back = p2.load_pathset("hallway_box_o3.h5")
print("receivers:", len(back.rx), " total paths:", back.length.size)
print("paths per Rx: min", back.n_paths().min(), " max", back.n_paths().max())
print("meta:", back.meta["scene"], back.meta["seed"], back.meta["max_order"])
grid = back.rx.reshape(len(ys), len(xs), 2)       # back to (ny, nx, 2) layout
print("grid shape:", grid.shape)
```

Output:

```
receivers: 861  total paths: 8655
paths per Rx: min 6  max 14
meta: hallway_box 1 3
grid shape: (41, 21, 2)
```

Example 3: a receiver array and differential path lengths

An array is just a set of receiver points. Here a 4-element half-wavelength ULA at 5.8 GHz is placed behind the obstacle, and the first-arrival length difference between elements is printed (the quantity that sets inter-element phase offsets later).

```
import numpy as np
import paths2d as p2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)
lam = p2.C0 / 5.8e9                                  # wavelength at 5.8 GHz (m)
array = p2.make_ula(center=(14.0, 1.5), spacing=lam / 2, n=4)   # along +x
ps = p2.trace_paths(scene, (2.0, 1.5), array, max_order=3)

first = np.array([ps.length[ps.rx_slice(i)][0] for i in range(4)])
print("element x (m):", np.round(array[:, 0], 4))
print("first-arrival length minus element 0 (mm):", np.round((first - first[0]) * 1e3, 2))
print("paths per element:", ps.n_paths())
```

Output:

```
element x (m): [13.9612 13.9871 14.0129 14.0388]
first-arrival length minus element 0 (mm): [ 0.   25.07 50.14 75.22]
paths per element: [14 14 16 16]
```

Example 4: reading per-receiver data from the flat arrays

Rows of every per-path array for receiver `r` are `ps.rx_slice(r)`. Use it to compute excess delays, filter by reflection order, or inspect which walls and angles a path used.

```
import numpy as np
import paths2d as p2

scene = p2.rectangle_room(10.0, 8.0)
ps = p2.trace_paths(scene, (2.0, 3.0), [[8.0, 5.0], [3.0, 6.0]], max_order=4)

for r in range(len(ps.rx)):
    sl = ps.rx_slice(r)                          # this receiver's rows
    L, order = ps.length[sl], ps.order[sl]
    excess_ns = (L - L[0]) / p2.C0 * 1e9         # relative to first arrival
    print(f"rx {r}: {L.size} paths, {(order <= 2).sum()} with order <= 2, "
          f"max excess delay {excess_ns.max():.1f} ns")
    print("   paths per order:", np.bincount(order))
    print("   2nd path walls:", ps.walls[sl][1], "(-1 = padding)")
    print("   2nd path incidence (deg):", np.round(np.degrees(ps.inc_angle[sl][1]), 1))
```

Output:

```
rx 0: 41 paths, 13 with order <= 2, max excess delay 132.5 ns
   paths per order: [ 1  4  8 12 16]
   2nd path walls: [ 0 -1 -1 -1] (-1 = padding)
   2nd path incidence (deg): [36.9  nan  nan  nan]
rx 1: 41 paths, 13 with order <= 2, max excess delay 126.6 ns
   paths per order: [ 1  4  8 12 16]
   2nd path walls: [ 3 -1 -1 -1] (-1 = padding)
   2nd path incidence (deg): [31. nan nan nan]
```

Example 5: plotting the paths of chosen receivers

`plot_paths` takes a receiver index and an optional Axes. Red is the direct path, blue are reflections. The second receiver sits in the box's shadow, so it has no red line.

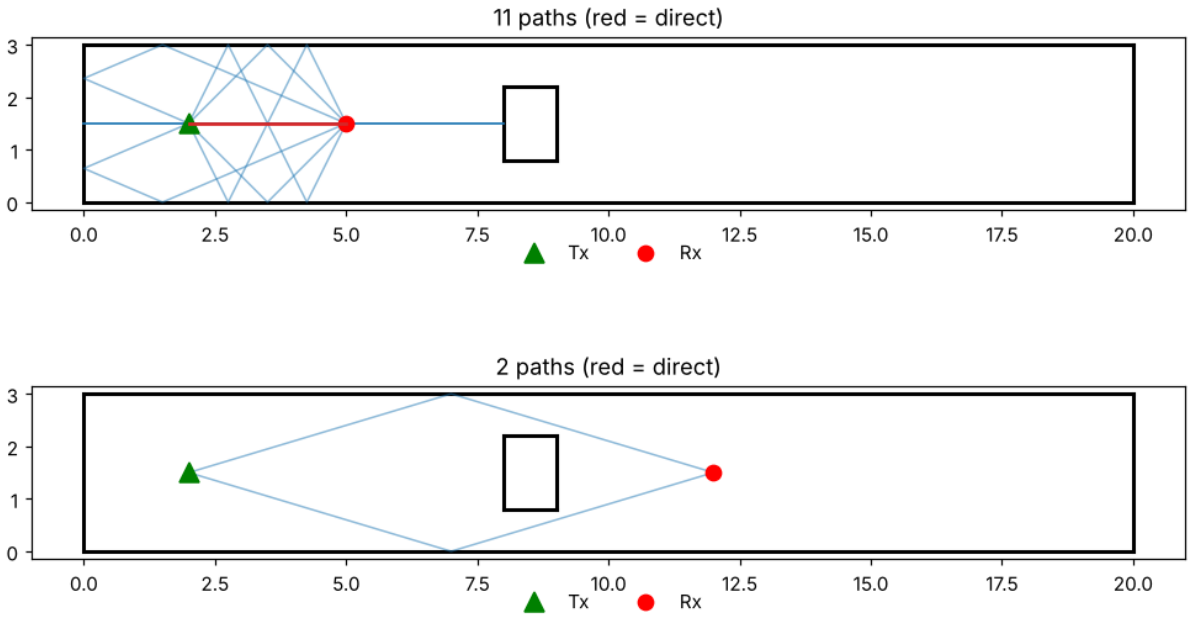
```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import paths2d as p2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)
rx = np.array([[5.0, 1.5], [12.0, 1.5]])                  # in LOS, and in the shadow
ps = p2.trace_paths(scene, (2.0, 1.5), rx, max_order=2)

fig, axes = plt.subplots(2, 1, figsize=(9, 5.5))
for i, ax in enumerate(axes):
    p2.plot_paths(ps, i, ax=ax)
fig.tight_layout()
fig.savefig("two_receivers.png", dpi=110)
print("direct path present:", [bool((ps.order[ps.rx_slice(i)] == 0).any()) for i in range(2)])
```

Output:

```
direct path present: [True, False]
```



Example 6: non-rectangular rooms, partitions and polygon obstacles

`rectangle_room` only makes rectangles. For other outlines start from an empty scene and add the outline as a polygon of `kind="room"`. (The empty-scene helper is private, `_empty_scene`, but stable for this use.) `free_space_mask` then respects the outline. In the output below, receiver 0 is far down the other arm: getting around the corner takes many steep bounces, so it has no path within 3 bounces (a real corner shadow, not an error). Receiver 3 is inside the triangle, so it is masked out.

```
import paths2d as p2

sc = p2._empty_scene(["wall"])                              # no walls yet
sc = p2.add_polygon(sc, [[0, 0], [12, 0], [12, 3], [3, 3], [3, 10], [0, 10]], kind="room")  # L corridor
sc = p2.add_wall(sc, (6.0, 0.0), (6.0, 1.2))                # partition stub from the bottom wall
sc = p2.add_polygon(sc, [[7.5, 1.5], [8.5, 1.5], [8.0, 2.4]])   # triangular obstacle

tx = (1.5, 8.0)
rx = [[10.0, 1.5], [1.5, 2.0], [2.0, 9.0], [8.0, 1.8]]   # far arm, corner, same arm, in triangle
print("legal (tx + rx):", p2.free_space_mask(sc, [tx] + rx).tolist())
ps = p2.trace_paths(sc, tx, rx, max_order=3)
print("paths per Rx:", ps.n_paths().tolist())
print("direct path present:", [bool((ps.order[ps.rx_slice(i)] == 0).any()) for i in range(4)])
```

Output:

```
legal (tx + rx): [True, True, True, True, False]
paths per Rx: [0, 24, 23, 0]
direct path present: [False, True, True, False]
```

Example 7: picking lambda/4 subsets from one fine grid

Tracing is frequency-independent, so trace one fine metre-spaced row and choose a stride per band. The printed actual spacing shows the rounding error of the stride; reduce `step` if that error matters.

```
import numpy as np
import paths2d as p2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)
step = 0.001                                                 # 1 mm grid
rx, xs, ys = p2.make_rx_grid((12.0, 12.5), (1.5, 1.5), step)   # one row, 501 receivers
ps = p2.trace_paths(scene, (2.0, 1.5), rx, max_order=3)

for name, f in {"5.8 GHz": 5.8e9, "28 GHz": 28e9}.items():
    lam = p2.C0 / f
    stride = max(int(round(lam / 4 / step)), 1)             # grid points per lambda/4
    pick = np.arange(0, len(rx), stride)                    # receiver indices to use
    print(f"{name}: lambda/4 = {lam / 4 * 1e3:.2f} mm, stride {stride}, "
          f"actual {stride * step * 1e3:.1f} mm, {len(pick)} receivers")
```

Output:

```
5.8 GHz: lambda/4 = 12.92 mm, stride 13, actual 13.0 mm, 39 receivers
28 GHz: lambda/4 = 2.68 mm, stride 3, actual 3.0 mm, 167 receivers
```

Example 8: geometric pruning with max_length

Dropping long paths shrinks the output and the later channel computation. The limit is in metres of total path length.

```
import paths2d as p2

scene = p2.rectangle_room(10.0, 8.0)
rx = [[8.0, 5.0]]
full = p2.trace_paths(scene, (2.0, 3.0), rx, max_order=5)
cut = p2.trace_paths(scene, (2.0, 3.0), rx, max_order=5, max_length=25.0)
print("all paths:", full.n_paths()[0], " longest:", round(float(full.length.max()), 1), "m")
print("max_length=25 m keeps:", cut.n_paths()[0], "paths")
```

Output:

```
all paths: 61  longest: 50.0 m
max_length=25 m keeps: 22 paths
```

Example 9: reading the HDF5 file directly with h5py

For use outside this module (MATLAB, other Python projects) the file is plain HDF5. Receiver `r`'s paths are rows `offsets[r]:offsets[r+1]` of every dataset under `/paths`.

```
import h5py
import numpy as np
import paths2d as p2

scene = p2.rectangle_room(10.0, 8.0)
p2.save_pathset("tiny.h5", p2.trace_paths(scene, (2, 3), [[8, 5]], max_order=2))

with h5py.File("tiny.h5", "r") as f:
    print("format:", f.attrs["format_version"])
    print("groups:", list(f.keys()))
    print("/paths datasets:", list(f["paths"].keys()))
    print("lengths (m):", np.round(f["paths/length"][:5], 3))
    offs = f["rx/offsets"][:]
    print("receiver 0 owns rows", offs[0], "to", offs[1] - 1)
```

Output:

```
format: 1.0
groups: ['paths', 'rx', 'scene', 'tx']
/paths datasets: ['aoa', 'aod', 'inc_angle', 'length', 'order', 'points', 'walls']
lengths (m): [ 6.325 10.    10.    10.198 10.198]
receiver 0 owns rows 0 to 12
```

Example 10: batch runs over several transmitter positions

One file per run, with the varied parameter recorded in the name and in `meta`.

```
import paths2d as p2

scene = p2.add_box(p2.rectangle_room(20.0, 3.0), 8.0, 0.8, 9.0, 2.2)
rx, _, _ = p2.make_rx_grid((10.0, 11.0), (1.0, 2.0), 0.1)
for tx_x in (1.0, 2.0, 3.0):                                  # sweep the transmitter
    ps = p2.trace_paths(scene, (tx_x, 1.5), rx, max_order=3, meta={"tx_x": tx_x})
    name = f"hall_tx{tx_x:.1f}_o3.h5"
    p2.save_pathset(name, ps)
    print(name, "paths:", ps.length.size)
```

Output:

```
hall_tx1.0_o3.h5 paths: 1246
hall_tx2.0_o3.h5 paths: 1040
hall_tx3.0_o3.h5 paths: 912
```

Limits and pitfalls

Topic	What to know
Physics	Specular reflection only. No diffraction, diffuse scattering or transmission, so regions behind obstacles are hard-edged shadows with only reflected paths.
Dimension	2D top-down. Floor and ceiling bounces and polarization are absent. Amplitudes built later should use 1/d spreading so they carry over to 3D.
Frequency	Nothing in the file depends on frequency; material indices are stored but not used. Reflection coefficients come from stage 2.
Points on walls	A Tx or Rx exactly on a wall, or on the extension of a wall, can give ambiguous results. Keep them clear of walls.
Collinear legs	A leg lying exactly along a wall is not treated as blocked (a measure-zero case).
Scene growth	Many segments with a high order produce a huge image set. Use max_order 3 or less, or max_length, for cluttered scenes.
Immutability	Forgetting to assign the result of `add_box` and friends leaves the scene unchanged.
Grid rounding	Floating-point grids may not land exactly on a target row or column. Pick receivers by nearest distance, not equality.
Versioning	Bump FORMAT_VERSION if the HDF5 layout changes. Old files with another major number raise ValueError.